

star

Figure 1: star

Single Star

In this program, you'll animate a single asterisk causing it to bounce back and forth from column zero to some column (currently defined as 40).

You'll learn:

- how to force output to happen even before a new line character
- how to overwrite a line on the console (rather than advancing to a new line)
- how to delay your program's execution for a short while (for timing purposes)
- usage of `cinttypes` which makes the specific flavor of an integer unambiguous

Write a program that animates a single asterisk like this:

Don't worry about the program terminating - just kill it when you're done looking.

No data structure is needed for this program as everything you need to do is based on a counter.

You'll need these include files:

```
#include <iostream>
#include <iomanip>
#include <cinttypes>
#include <chrono>
#include <thread>
```

I used the features of `iomanip` to space out the animating asterisk.

`cinttypes` contains aliases for integer types that are unambiguous. That is, when you say `int`, how big is it? Depends. Using `cinttypes` you can declare:

type	meaning	without <code>cinttypes</code>
<code>uint8_t</code>	unsigned 8 bit integer	unsigned char
<code>uint16_t</code>	unsigned 16 bit integer	unsigned short
<code>uint32_t</code>	unsigned 32 bit integer	unsigned int
<code>uint64_t</code>	unsigned 64 bit integer	unsigned long
<code>int8_t</code>	signed 8 bit integer	char
<code>int16_t</code>	signed 16 bit integer	short
<code>int32_t</code>	signed 32 bit integer	int
<code>int64_t</code>	signed 64 bit integer	long

`chrono` and `thread` are used together to implement delays in your program. Without pausing your program, it will update the screen so quickly, you won't be able to enjoy the animation.

For example:

```
this_thread::sleep_for(chrono::milliseconds(16));
```

will cause your program to sleep for at least 16 milliseconds but maybe a tad more. The value 16 milliseconds is chosen as it is 1/60th of a second.

Output without new lines

You're used to this:

```
cout << "Foo" << endl;
```

The `endl` is doing two things for you:

1. Of course, it's giving you a new line but it is also
2. Triggering the output to actually render on your console

Console output is buffered for efficiency. Actual output only happens when new lines are emitted. In this program, we're not using new lines at all. Instead, after text is output, we'll emit only a carriage return (`'\n'`).

To force output (without a new line), do:

```
cout.flush();
```

Source code

DO NOT LOOK AT THIS UNTIL YOU HAVE TRIED TO WRITE THE CODE YOURSELF! With that said, don't feel bad about taking a peek and reading the comments.

Related projects

- Cylon Effect
- Walkies